

Pilas

Data Structures

Carlos Rojas-Sánchez

Universidad del Mar

Contenido

1. Representación de una pila

2. Operaciones con pilas

3. Notación infija, prefija y postfija

4. Aplicaciones de las pilas

Representación de una pila

¿Qué es una Pila (Stack)?

Una **pila** es una estructura de datos lineal que sigue el principio **LIFO** (*Last In, First Out*: El último en entrar es el primero en salir).

La Metáfora de la Pila de Platos

- Los platos se colocan uno encima de otro.
- Solo puedes agregar (**apilar**) un plato nuevo en la parte superior.
- Solo puedes retirar (**desapilar**) el plato que está hasta arriba.

```
Cima (Top) ⟶ [ Plato 3 ]
             [ Plato 2 ]
             [ Plato 1 ]
```

Operaciones Principales de una Pila

El acceso a los elementos está estrictamente restringido al extremo superior denominado **Cima** (*Top*).

Operaciones Esenciales

- **`push(elemento)`:** Inserta un nuevo elemento en la cima de la pila.
- **`pop()`:** Extrae y elimina el elemento ubicado en la cima.
- **`peek()` / `top()`:** Consulta el valor de la cima sin eliminarlo.
- **`isEmpty()`:** Verifica si la pila se encuentra vacía.

Formas de Representación en Memoria

Una pila puede implementarse computacionalmente de dos maneras principales:

1. **Representación Estática (Arreglos)**
   - Utiliza un arreglo unidimensional de tamaño fijo.
   - Requiere una variable entera para llevar el índice de la `cima`.
   - **Riesgo:** Desbordamiento si se excede la capacidad máxima (*Stack Overflow*).

2. **Representación Dinámica (Listas Enlazadas)**
   - Utiliza nodos en memoria dinámica (*Heap*).
   - La cima de la pila corresponde a la **cabeza** de la lista enlazada.
   - Tamaño flexible según la memoria disponible.

Definición en C: Pila Estática

Implementación usando una estructura con un arreglo y un control de índice:

```
#define MAX 100

typedef struct {
  int elementos[MAX];
  int cima; // Índice del último elemento (-1 si está vacía)
} PilaEstatica;

void inicializar(PilaEstatica *p) {
  p->cima = -1;
}
```

Definición en C: Pila Dinámica

Implementación utilizando una lista enlazada simple:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct NodoPila {
  int dato;
  struct NodoPila *siguiente;
} NodoPila;

// La cima es un puntero al primer nodo de la lista
typedef struct {
  NodoPila *cima;
} PilaDinamica;
```

Ventaja: No desperdicia memoria estática y crece dinámicamente en tiempo de ejecución.

Representación Gráfica: Pila Dinámica

Estructura visual de una pila enlazada con tres elementos:

```
Pila.cima ⟶ [ 30 | * ] ⟶ [ 20 | * ] ⟶ [ 10 | NULL ]
```

- El nodo `30` es el último en llegar y el primero en salir (Cima).
- Cada nuevo elemento se inserta al inicio de la estructura para lograr un costo temporal de $\mathcal{O}(1)$.

Operaciones con pilas

Operaciones Fundamentales

Las operaciones en una pila gestionan los elementos exclusivamente a través de la **cima** (*top*), garantizando el comportamiento LIFO.

Operaciones Esenciales

- **Push:** Inserta un elemento en la cima.
- **Pop:** Extrae y elimina el elemento de la cima.
- **Peek / Top:** Consulta el valor de la cima sin eliminarlo.
- **isEmpty:** Comprueba si la pila no contiene elementos.

Eficiencia

Todas las operaciones básicas de una pila implementada con listas enlazadas tienen una complejidad temporal de $\mathcal{O}(1)$.

Paso a Paso: Operación Push (Apilar)

Agregar un elemento a la cima de la pila dinámica requiere:

Secuencia de Inserción

1. **Reservar Memoria:** Crear dinámicamente un nuevo nodo en el *Heap*.
2. **Asignar Valor:** Guardar el dato en el campo correspondiente del nodo.
3. **Enlazar:** Hacer que el puntero `siguiente` del nuevo nodo apunte a la `cima` actual.
4. **Actualizar Cima:** Redirigir el puntero `cima` de la pila para que señale al nuevo nodo.

Código: Operación Push

Implementación de la inserción en la cima utilizando una pila dinámica:

```
void push(PilaDinamica *pila, int valor) {
    NodoPila *nuevo = (NodoPila*)malloc(sizeof(NodoPila));
    if (nuevo == NULL) {
        printf("Error: Stack Overflow (Memoria llena)\n");
        return;
    }
    nuevo->dato = valor;
    nuevo->siguiente = pila->cima; // Enlazar a la cima anterior
    pila->cima = nuevo;            // Actualizar la cima
}
```

Complejidad Temporal: $\mathcal{O}(1)$ (Constante).

Paso a Paso: Operación Pop (Desapilar)

Extraer el elemento de la cima de manera segura:

Secuencia de Extracción

1. **Validación:** Verificar que la pila no esté vacía (`cima != NULL`) para evitar errores de tipo *Underflow*.
2. **Guardar Referencia:** Asignar un puntero temporal al nodo de la cima actual (`temp = pila->cima`).
3. **Mover Cima:** Avanzar el puntero `cima` al siguiente nodo (`pila->cima = pila->cima->siguiente`).
4. **Liberar y Retornar:** Guardar el valor del nodo extraído, liberar la memoria con `free(temp)` y devolver el dato.

Código: Operación Pop

Implementación de la extracción con manejo de pila vacía:

```
int pop(PilaDinamica *pila) {
    if (pila->cima == NULL) {
        printf("Error: Stack Underflow (Pila vacia)\n");
        return -1; // Valor de error convencional
    }
    NodoPila *temp = pila->cima;
    int valor_extraido = temp->dato;

    pila->cima = temp->siguiente; // Mover cima al siguiente nodo
    free(temp);                   // Liberar memoria del nodo viejo

    return valor_extraido;
}
```

Complejidad Temporal: $\mathcal{O}(1)$ (Constante).

Operaciones Auxiliares: Peek e isEmpty

Funciones complementarias para consultar el estado de la pila:

1. Verificar si está vacía (`isEmpty`):

```
int isEmpty(PilaDinamica *pila) {
    return (pila->cima == NULL); // Retorna 1 si esta vacia, 0 si tiene elementos
}
```

2. Consultar la Cima sin extraer (peek):

```
int peek(PilaDinamica *pila) {
    if (isEmpty(pila)) {
        printf("La pila esta vacia.\n");
        return -1;
    }
    return pila->cima->dato; // Retorna el valor del tope
}
```

Notación infija, prefija y postfija

Introducción a las Notaciones Aritméticas

La forma en que escribimos y evaluamos expresiones aritméticas depende de la posición relativa entre los **operandos** (variables o números) y los **operadores** ($+, -, *, /$).

Las Tres Notaciones Principales

- **Infija:** El operador se encuentra *entre* los operandos ($A + B$).
- **Prefija (Polaca):** El operador precede *antes* de los operandos ($+AB$).
- **Postfija (Polaca Inversa):** El operador va *después* de los operandos ($AB+$).

Objetivo

Las computadoras procesan con extrema eficiencia las expresiones postfijas utilizando una **pila**.

1. Notación Infija

Es la notación tradicional que utilizamos los seres humanos en el álgebra cotidiana.

Características

- Los operadores binarios se colocan entre los operandos: `A + B * C`.
- **Desventaja computacional:** Requiere reglas estrictas de **precedencia de operadores** y el uso obligatorio de **paréntesis** para alterar el orden $((A + B) * C)$.
- Analizar sintácticamente una expresión infija directamente en código es complejo y costoso para una computadora.

2. Notación Prefija (Polaca)

Propuesta por el matemático polaco Jan Łukasiewicz en 1924.
Los operadores se escriben antes de sus operandos.

Características

- Ejemplo: La expresión infija $A + B$ se escribe como $+$ A B.
- Ejemplo complejo: $(A + B) * C \implies * + \text{A B C}$.
- **Ventaja clave:** No requiere paréntesis ni reglas de precedencia; el orden de evaluación es completamente inequívoco escaneando de derecha a izquierda (o con árboles sintácticos).

3. Notación Postfija (Polaca Inversa - RPN)

Los operadores se escriben después de sus operandos correspondientes. Es la forma nativa en la que operan las arquitecturas basadas en pilas.

Características

- Ejemplo: $A + B \implies$ A B+
- Ejemplo complejo: $(A + B) * C \implies$ A B + C*
- Ejemplo con múltiples operadores:
  $A + B * C - D \implies$ A B C * +D−
- **Ventaja clave:** Elimina totalmente la necesidad de paréntesis y prioridades de operadores.

¿Por qué las Pilas prefieren la Notación Postfija?

Las máquinas de pila (como las máquinas virtuales de Java o intérpretes) procesan expresiones postfijas de manera lineal y directa.

Compatibilidad Natural con LIFO

- Cuando la computadora lee un operando, lo **apila**.
- Cuando lee un operador, **desapila** los operandos necesarios, realiza la operación matemática y vuelve a apilar el resultado parcial.
- Todo el proceso se realiza en una sola pasada lineal ($\mathcal{O}(n)$) sin ambigüedades.

Paso a Paso: Evaluación de una Expresión Postfija

Algoritmo para evaluar una cadena en notación postfija usando una Pila:

Reglas de Evaluación

1. Recorrer la expresión de izquierda a derecha símbolo por símbolo.
2. **Si es un operando:** Convertirlo a número y hacer **push** en la pila.
3. **Si es un operador:**
   - Hacer **pop** para obtener el segundo operando (Op_2).
   - Hacer **pop** para obtener el primer operando (Op_1).
   - Calcular Op_1 [operador]Op_2.
   - Hacer **push** del resultado obtenido en la pila.
4. Al finalizar, el único valor restante en la pila es el resultado final.

Código: Evaluación de Postfix

Implementación en C utilizando una pila dinámica de enteros:

```
int evaluar_postfix(char *expresion) {
    PilaDinamica pila;
    inicializar_pila(&pila);

    for (int i = 0; expresion[i] != '\0'; i++) {
        char c = expresion[i];
        if (c == ' ' || c == '\t') continue;

        if (isdigit(c)) {
            push(&pila, c - '0'); // Convertir char a int
        } else {
            int op2 = pop(&pila);
            int op1 = pop(&pila);
            switch (c) {
                case '+': push(&pila, op1 + op2); break;
                case '-': push(&pila, op1 - op2); break;
                case '*': push(&pila, op1 * op2); break;
                case '/': push(&pila, op1 / op2); break;
            }
        }
    }
    return pop(&pila); // Resultado final
}
```

Paso a Paso: Conversión de Infija a Postfija (Shunting-Yard)

Para convertir una expresión de humano (infija) a máquina (postfija) se utiliza el algoritmo de la estación de clasificación:

Algoritmo con Pila Auxiliar de Operadores

1. **Operando:** Se imprime/agrega directamente al resultado final.
2. **Paréntesis abierto '(':** Se hace **push** en la pila.
3. **Paréntesis cerrado ')':** Desapilar operadores hacia el resultado hasta encontrar el '('.
4. **Operador ($+, -, *, /$):** Desapilar de la pila todos los operadores que tengan **mayor o igual precedencia** y pasarlos al resultado, luego hacer **push** del operador actual.

Ejemplo de Conversión: $A + B * C$

Seguimiento paso a paso del algoritmo de conversión:

Símbolo leído	Estado de la Pila	Salida (Postfix)
A	(Vacía)	A
+	+	A
B	+	A B
*	+, *	A B
C	+, *	A B C
(Fin)	(Vacía)	A B C * +

Resultado final en postfija: `A B C * +`

Código: Conversión Infija a Postfija

Función auxiliar para determinar la jerarquía de operadores:

```
int precedencia(char operador) {
    if (operador == '+' || operador == '-') return 1;
    if (operador == '*' || operador == '/') return 2;
    return 0;
}

void infija_a_postfija(char *infija, char *postfija) {
    // Implementacion utilizando pila auxiliar de caracteres
    // y reglas de precedencia para vaciar operadores.
    // (Estructura base usada en compiladores y calculadoras)
}
```

Aplicación Directa: Permite a cualquier lenguaje compilar operaciones matemáticas complejas interpretando el orden correcto de ejecución.

Aplicaciones en Arquitectura y Software

Las notaciones postfijas y prefijas son pilares fundamentales en la informática moderna:

Casos de Uso Reales

- **Calculadoras Científicas HP:** Utilizan RPN (Postfix) de forma nativa por su velocidad operativa sin paréntesis.
- **Compiladores e Intérpretes:** Transforman el código fuente en árboles de sintaxis abstracta (AST) y código de tres direcciones o bytecode evaluado por pilas virtuales.
- **Máquinas Virtuales (JVM / Python):** Las operaciones a nivel de bytecode operan bajo arquitectura basada en pila (p.ej., instrucciones `iload`, `iadd`, `istore`).

Aplicaciones de las pilas

Aplicaciones Prácticas de las Pilas

Debido a su comportamiento LIFO (*Last In, First Out*), las pilas son indispensables en la arquitectura de computadoras, compiladores y algoritmos avanzados.

Áreas de Aplicación Principal

- **Gestión de Llamadas a Funciones (Call Stack):** El sistema operativo rastrea el flujo de ejecución y retorno de funciones.
- **Validación de Sintaxis y Delimitadores:** Comprobación de paréntesis, llaves y corchetes balanceados en compiladores.
- **Historiales y Funciones Undo/Redo:** Control de estados en editores de texto y navegadores.
- **Algoritmos de Búsqueda y Grafos:** Recorridos en profundidad (DFS - *Depth-First Search*).

1. Validación de Delimitadores Balanceados

Los compiladores y editores de código verifican que cada llave {}, corchete [] o paréntesis () abierto tenga su respectivo cierre en orden correcto.

Algoritmo con Pila

1. Recorrer la cadena carácter por carácter.
2. Si se encuentra un delimitador de apertura (`'('`, `''`, `'['`), hacer push en la pila.
3. Si se encuentra un delimitador de cierre (`')'`, `''`, `']'`):
   - Si la pila está vacía, hay un error (falta apertura).
   - Si no está vacía, hacer **pop** y verificar que coincida con el tipo correspondiente.
4. Al finalizar el recorrido, la pila debe quedar completamente vacía.

Código: Validación de Paréntesis Balanceados

```
int validar_parentesis(char *expresion) {
    PilaDinamicaChar pila;
    inicializar_pila_char(&pila);

    for (int i = 0; expresion[i] != '\0'; i++) {
        char c = expresion[i];
        if (c == '(' || c == '{' || c == '[') {
            push_char(&pila, c);
        } else if (c == ')' || c == '}' || c == ']') {
            if (isEmptyChar(&pila)) return 0; // Error: falta apertura
            char tope = pop_char(&pila);
            if ((c == ')' && tope != '(') ||
                (c == '}' && tope != '{') ||
                (c == ']' && tope != '[')) {
                return 0; // Error: no coinciden
            }
        }
    }
    int resultado = isEmptyChar(&pila); // 1 si está vacía, 0 si sobra algo
    return resultado;
}
```

2. Pila de Llamadas (Call Stack) en Programación

Cuando un programa en C ejecuta una función, el procesador utiliza una sección de memoria llamada **Pila de Llamadas** (*Call Stack*).

Funcionamiento Interno

- Cada vez que se llama a una función, se crea un **registro de activación** (*stack frame*) que almacena sus variables locales y la dirección de retorno.
- Al finalizar la función, el registro se desapila automáticamente y el flujo regresa al punto exacto donde fue llamada.
- **Stack Overflow:** Ocurre si hay recursión infinita y se agota el espacio de la pila reservado por el sistema operativo.

3. Historiales y Mecanismo Undo / Redo

Las operaciones de retroceso en software dependen de estructuras de pila duales:

Implementación de Deshacer (Undo)

- Cada cambio de estado en un documento se guarda en una **Pila de Undo**.
- Al presionar *Ctrl+Z*, se hace **pop** de la pila de acciones y se revierte el estado.
- La acción deshecha suele moverse a una **Pila de Redo** por si el usuario desea repetir la acción.

$$\texttt{Pila Undo} \longleftrightarrow \texttt{Estado Actual} \longleftrightarrow \texttt{Pila Redo}$$

Resumen de Aplicaciones y Ventajas de las Pilas

Aplicación	Componente Clave	Objetivo
Expresiones Aritméticas	Pila de valores / operadores	Evaluar y convertir notación Postfix/Prefix.
Sintaxis y Compiladores	Pila de caracteres	Verificar paréntesis y llaves balanceadas.
Ejecución de Código	Pila de llamadas (*Call Stack*)	Gestionar contextos y retornos de funciones.
Editores de texto	Pilas duales (Undo/Redo)	Revertir y re-aplicar cambios históricos.